

# Chapter 2

## **Processes and Process Management**

# Processes

- A **process** is a program in execution.
- All modern computers do many things at the same time
- In a uni-processor system, at any instant, CPU is running only one process
- But in a multiprogramming system, CPU switches from processes quickly, running each for tens or hundreds of ms
- Sometimes called ***pseudoparallelism*** since one has the illusion of parallel processor.

# The process model

Even though in actuality there are many processes running at once, the OS gives each process the illusion that it is running alone.

- (a) **Virtual time** – the time used by just this processes.
- (b) **Virtual memory** – the memory as viewed by the processes.

examples of abstractions provided by the OS to the user to present a pleasant virtual machine.

# Process Creation

Events which can cause process creation

- System initialization.
- Execution of a process creation system call by a running process.
- A user request to create a new process.
- Initiation of a batch job.

# Process Termination

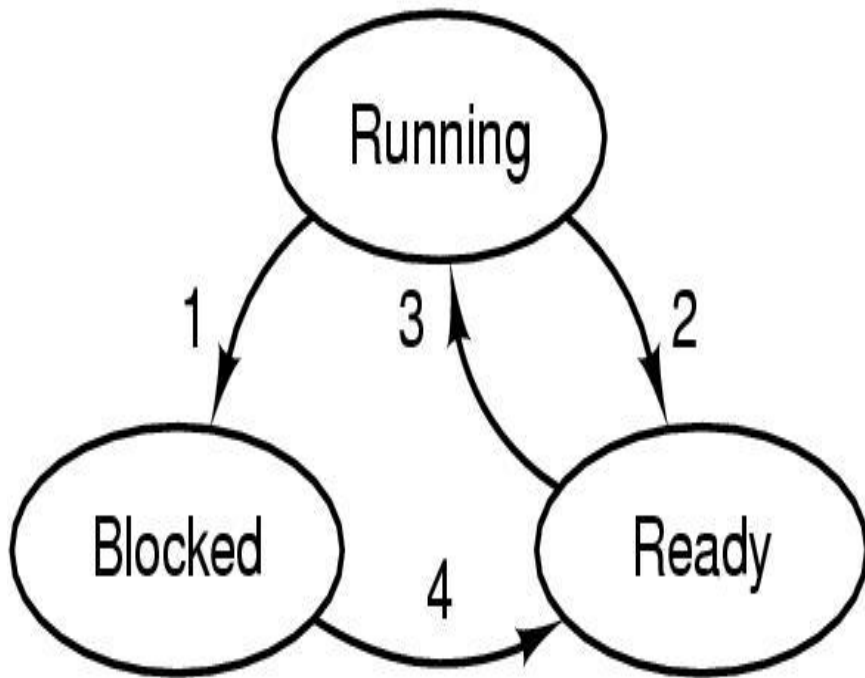
Events which cause process termination:

- Normal exit (voluntary).
- Error exit (voluntary).
- Fatal error (involuntary).
- Killed by another process (involuntary).

# Process Hierarchies

- Modern general purpose operating systems permit a user to **create** and **destroy** processes.
- In unix this is done by the **fork** system call, which creates a **child** process, and the **exit** system call, which terminates the current process.
- After a fork both parent and child **keep running** (indeed they have the *same* program text) and each can fork off other processes.
- A **process tree** results. The root of the tree is a special process created by the OS during startup.
- A process can **choose to wait** for children to terminate.

# Process States



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

A process can be in running, blocked, or ready state.  
Transitions between these states are as shown.

# Implementation of Processes

- The OS organizes the data about each process in a table naturally called the **process table**.
- Each entry in this table is called a **process table entry (PTE)** or **process control block**.
- One entry per process.
- The central data structure for process management.
- A process state transition is reflected by a change in the value of one or more fields in the PTE.

# Thread

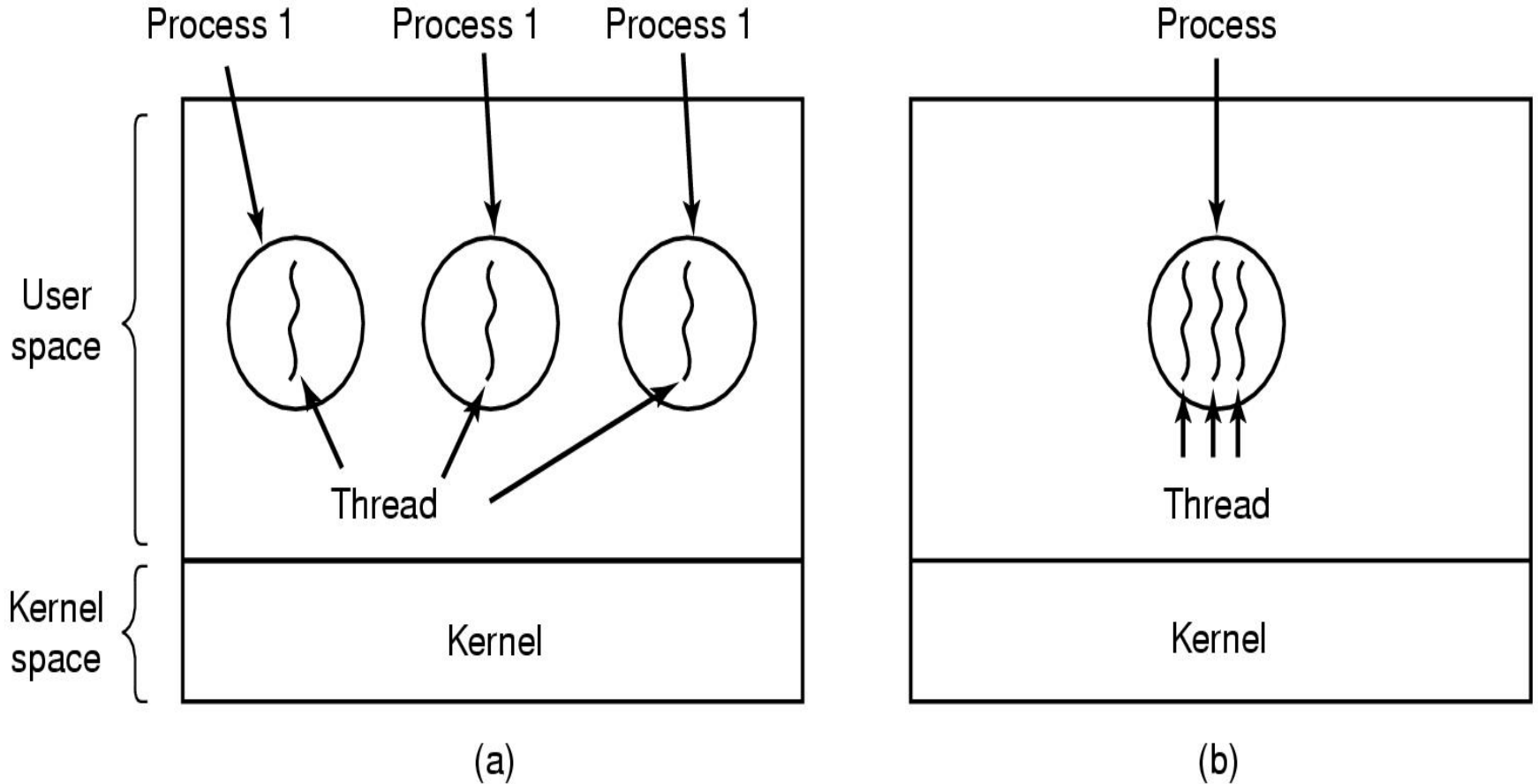
| <b>Per process items</b>    | <b>Per thread items</b> |
|-----------------------------|-------------------------|
| Address space               | Program counter         |
| Global variables            | Registers               |
| Open files                  | Stack                   |
| Child processes             | State                   |
| Pending alarms              |                         |
| Signals and signal handlers |                         |
| Accounting information      |                         |

- The first column lists some items shared by all threads in a process.
- The second one lists some items private to each thread.

# The Thread Model

- A process contains a number of **resources** such as address space, open files, accounting information, etc.
- In addition to these resources, a process has a **thread of control**.
- The idea of threads is to permit multiple threads of control to execute within one process.
- This is often called **multithreading** and threads are often called **lightweight processes**.

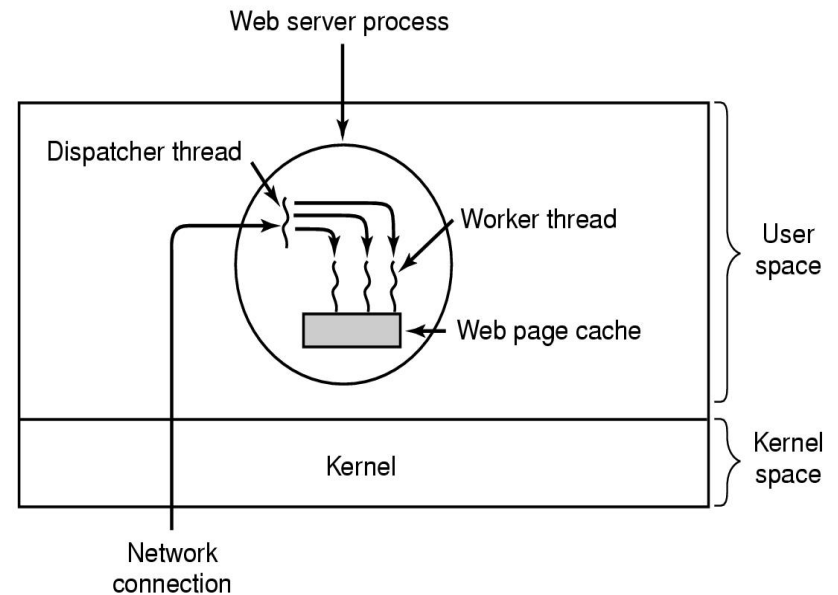
# The Classical Thread Model



- (a) Three processes each with one thread.
- (b) One process with three threads.

- Often, when a process A is blocked (say for I/O) there is still computation that can be done.
- Another process can't B do this computation since it doesn't have access to the A's memory.
- But **two threads in the same process do share the memory** so there is no problem.
- Each thread is responding to a single WWW connection.
- While one thread is blocked on I/O, another thread can be processing another WWW connection.

# Thread example



# Reasons to use threads

- Enables parallelism (web server) with blocking system calls
- Threads are faster to create and destroy than processes
- Natural for multiple cores
- Easy programming model

# Situations to use threads

- **Doing lengthy processing:** When a windows application is calculating it cannot process any more messages. As a result, the display cannot be updated.
- **Doing background processing:** Some tasks may not be time critical, but need to execute continuously.
- **Doing I/O work:** I/O to disk or to network can have unpredictable delays. Threads allow you to ensure that I/O latency does not delay unrelated parts of your application.

# Threads: Benefits

- **User responsiveness**
  - When one thread blocks, another may handle user I/O
  - But: depends on threading implementation
- **Resource sharing: economy**
  - Memory is shared (i.e., address space shared)
  - Open files, sockets shared
- **Speed**
  - E.g., Solaris: thread creation about 30x faster than heavyweight process creation; context switch about 5x faster with thread
- **Utilizing hardware parallelism**
  - Like heavy weight processes, can also make use of multiprocessor architectures

# Threads: Drawbacks

- **Synchronization**
  - Access to shared memory or shared variables must be controlled ***if the memory or variables are changed***
  - Can add complexity, bugs to program code
  - E.g., need to be very careful to avoid *race conditions*, *deadlocks* and other problems
- **Lack of independence**
  - Threads not independent, within a Heavy-Weight Process (HWP)
  - The RAM address space is shared; No memory protection from each other
  - The stacks of each thread are intended to be in separate RAM, but if one thread has a problem (e.g., with pointers or array addressing), it could write over the stack of another thread

# Process Scheduling

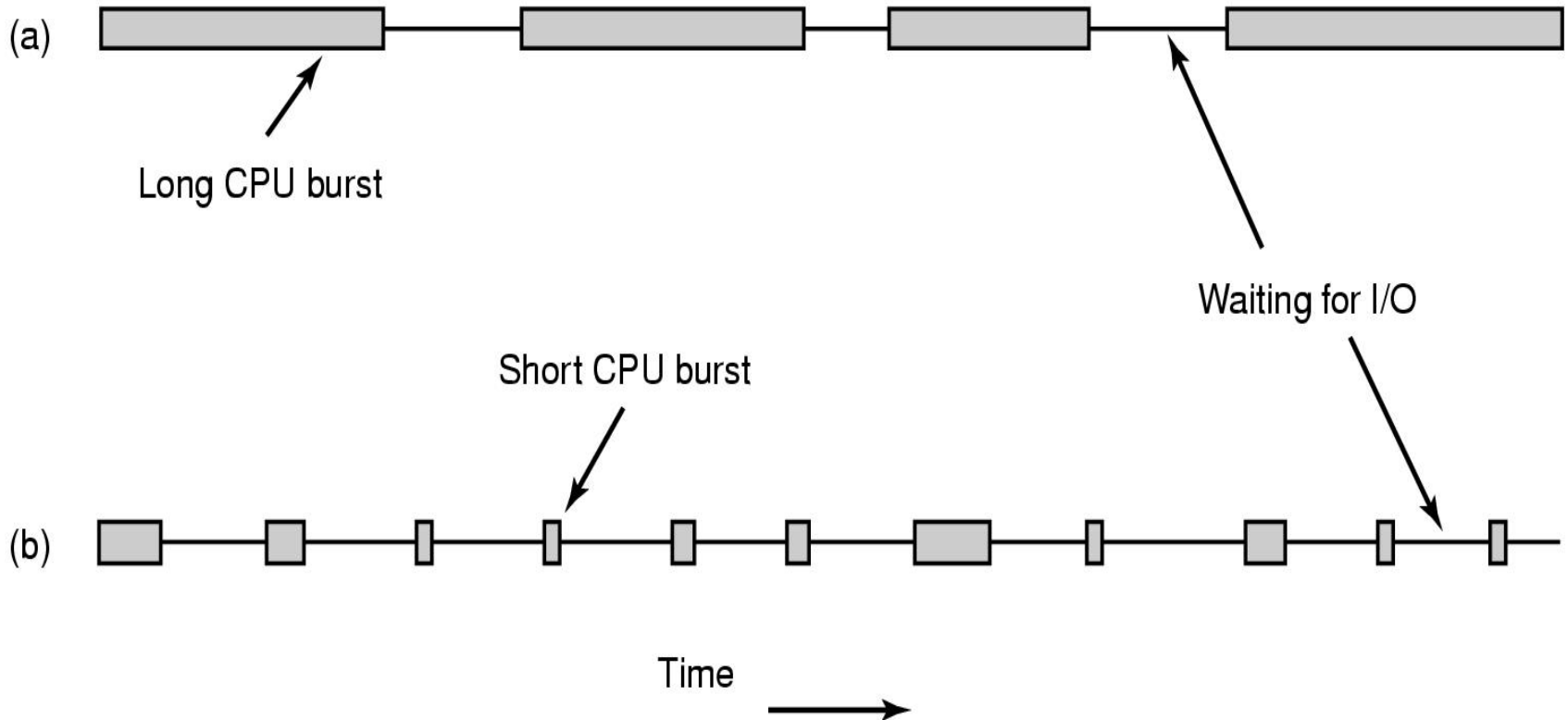
- Which process to run next?
- When a process is running, should CPU run to its end, or switch between different jobs?
- Scheduling the processor is often called ``**process scheduling**'' or simply ``scheduling''.

- The part of the operating system that makes this decision is called the **scheduler**.
- The algorithm it uses is called the **scheduling algorithm**.
- Scheduling may involve both processes and threads.

# Scheduling Criteria

- **Fairness** – each process must get its fair share of the CPU.
- **Throughput** – number of jobs which are completed per unit time.
- **Turnaround Time** – time between the submissions of a job to the time of completion.
- **Waiting Time** – time spent to get into memory and time spent waiting in the ready queue.
- **Response Time** – time from submission of the job until the first response is produced.

# Scheduling – Process Behavior



Bursts of CPU usage alternate with periods of waiting for I/O.

(a) A CPU-bound process. (b) An I/O-bound process.

# Concepts

- **Preemptive algorithm**
  - If a process is still running at the end of its time interval, it is suspended and another process is picked up
- **Non-preemptive**
  - Picks a process to run and then lets it run till it blocks or it voluntarily releases the CPU

# Categories of Scheduling Algorithms

Different environments need different scheduling algorithms

- **Batch**

- Still in wide use in business world
- Non-preemptive algorithms reduces process switches

- **Interactive**

- Preemptive is necessary

- **Real time**

- Processes run quickly and block

# Scheduling in Batch Systems

- First-come first-served
- Shortest job first
- Shortest Remaining Time  
Next

# Scheduling in Batch Systems

- **First-Come First-Served**

- Non-preemptive
- The CPU is assigned in the order processes required
- When the running process blocks the following one in the queue is selected
- When a blocked process becomes ready it is put on the end of the queue.
- Not optimal

- **Shortest Job First**

- Non-preemptive
- Suppose we know the run-time in advance
- The CPU is assigned to the shortest job in the queue
- Optimal if all the jobs are available at the same time.

- **Shortest Remaining Time Next**
  - Preemptive
  - The scheduler here chooses the process whose remaining run-time is the shortest.
  - The time has to be known in advance
  - New short jobs get good service

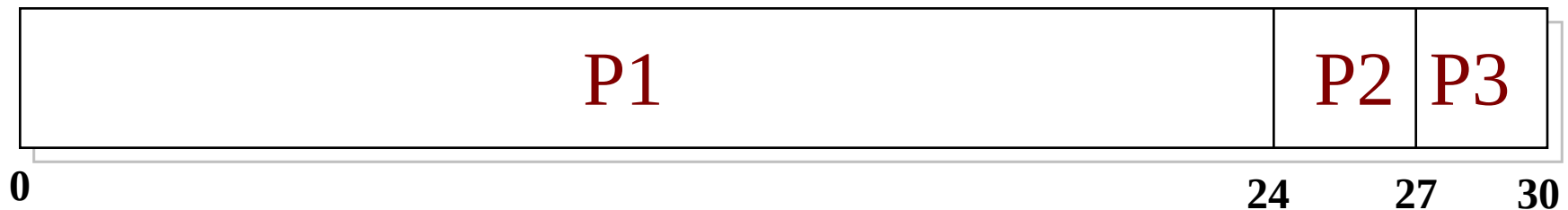
# **First-come-first-served (FCFS) scheduling**

- Processes get the CPU in the order they request it and run until they release it
- Ready processes form a FIFO queue

# FCFS may cause long waiting times

| <u>Process</u> | <u>Burst time (milli)</u> |
|----------------|---------------------------|
| P1             | 24                        |
| P2             | 3                         |
| P3             | 3                         |

If they arrive in the order P1, P2, P3, we get:



Average waiting time:  $(0+24+27) / 3 = 17$

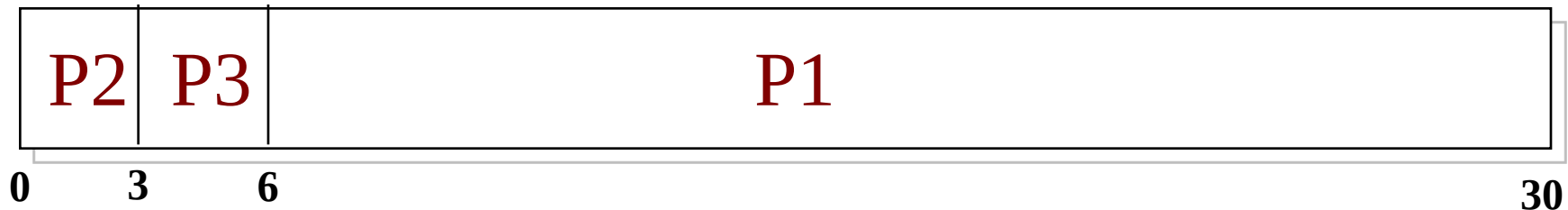
# FCFS may cause long waiting times (cont'd)

| Process | Burst time (milli) |
|---------|--------------------|
| P1      | 24                 |
| P2      | 3                  |
| P3      | 3                  |

If they arrive in the order P1, P2, P3, average waiting time **17**

---

What if they arrive in the order P2, P3, P1?

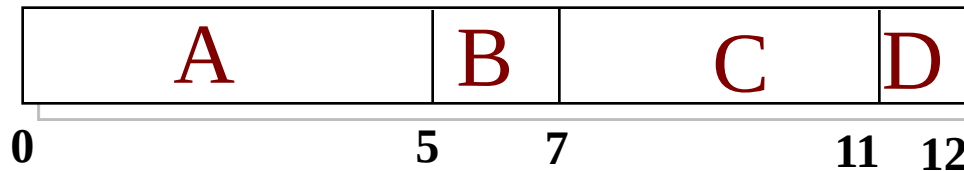


Average waiting time:  $(0+3+6) / 3 = 3$

# Shortest Job First (SJF) scheduling

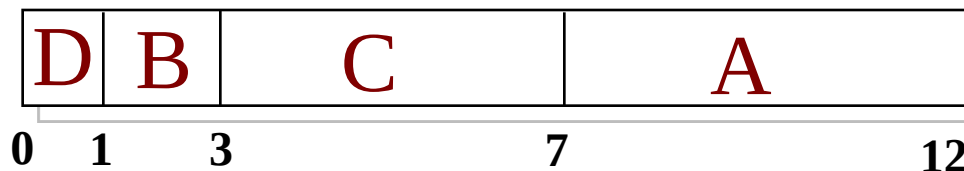
- ❑ The CPU is assigned to the process that has the smallest next CPU burst
- ❑ In some cases, this quantity is known or can be approximated

| Process | Burst time (milli) |
|---------|--------------------|
| A       | 5                  |
| B       | 2                  |
| C       | 4                  |
| D       | 1                  |



Average waiting time: 8.75

SJF schedule

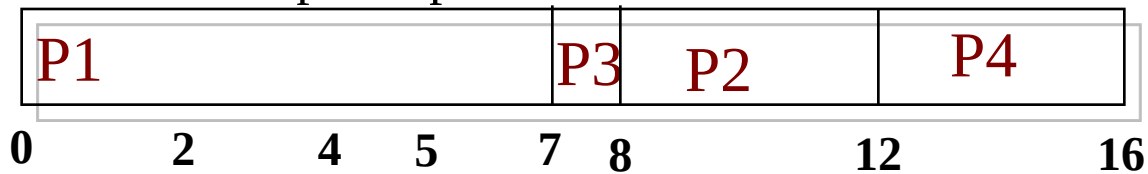


Average waiting time: 5.75

# Example: non-preemptive SJF

| Process | Arrival time | Burst time |
|---------|--------------|------------|
| P1      | 0            | 7          |
| P2      | 2            | 4          |
| P3      | 4            | 1          |
| P4      | 5            | 5          |

Non-preemptive SJF schedule



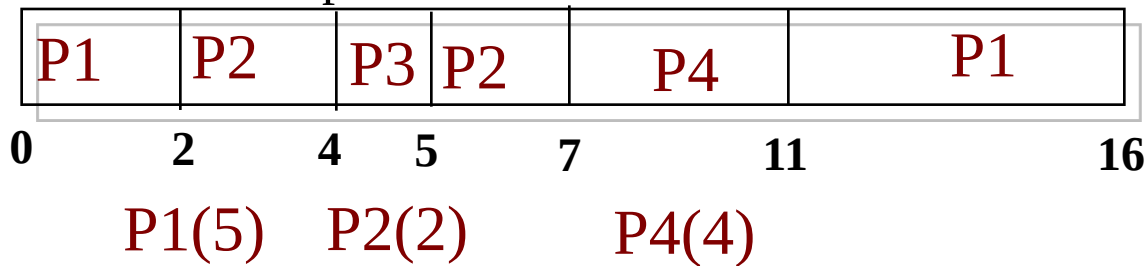
P2      P3 P4

Average waiting time:  $(0+6+3+7) / 4 = 4$

# Preemptive SJF (Shortest Remaining Time Next)

| Process | Arrival time | Burst time |
|---------|--------------|------------|
| P1      | 0            | 7          |
| P2      | 2            | 4          |
| P3      | 4            | 1          |
| P4      | 5            | 5          |

Preemptive SJF schedule



Average waiting time:  $(9+1+0+2) / 4 = 3$

# Scheduling in Interactive Systems

- Round-robin scheduling
- Priority scheduling

# Round Robin

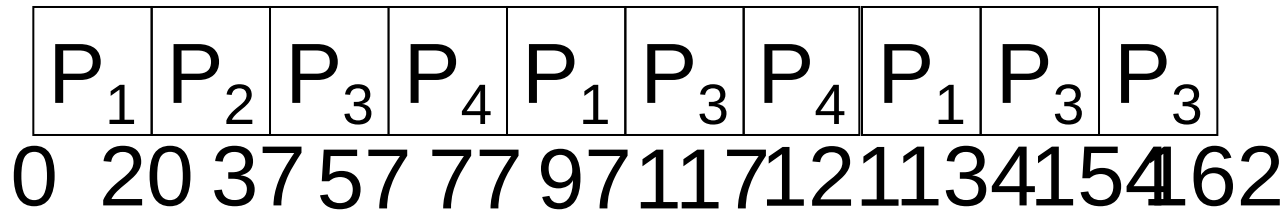
- ❑ Each process gets a small unit of CPU time (*time-quantum*), usually 10-100 milliseconds
- ❑ For  $n$  ready processes and time-quantum  $q$ , no process waits more than  $(n - 1)q$
- ❑ Approaches FCFS when  $q$  grows
- ❑ Time-quantum  $\sim$  switching time
  - relatively large waste of CPU time
- ❑ Time-quantum  $\gg$  switching time
  - long response (waiting) times, FCFS

## Example: RR with Time Quantum = 20

- Arrival time = 0
- Time quantum = 20

| <u>Process</u> | <u>Burst Time</u> |
|----------------|-------------------|
| P1             | 53                |
| P2             | 17                |
| P3             | 68                |
| p4             | 24                |

- The Gantt chart is



- Typically, higher average turnaround than SJF, but better response

# Size of time quantum

- The performance of RR depends heavily on the size of time quantum
- Time quantum
  - Too big, = FCFS
  - Too small:
    - Hardware: Process sharing
    - Software: context switch, high overhead, low CPU utilization
  - Must be large with respect to context switch

# Priority scheduling

- Each priority has a priority number
- The highest priority can be scheduled first
- If all priorities equal, then it is FCFS

# Example

- E.g.:

| <u>Process</u> | <u>Burst Time</u> | <u>Priority</u> |
|----------------|-------------------|-----------------|
| P1             | 10                | 3               |
| P2             | 1                 | 1               |
| P3             | 2                 | 3               |
| P4             | 1                 | 4               |
| P5             | 5                 | 2               |

- Priority (nonpreemptive)



- Average waiting time  
 $= (6 + 0 + 16 + 18 + 1) / 5 = 8.2$

# Introduction To Deadlocks

Deadlock can be defined formally as follows:

*A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause.*

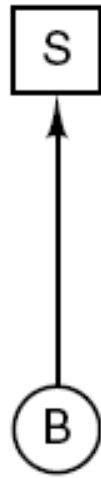
# Conditions for Resource Deadlocks

1. **Mutual exclusion condition** – only one process at a time can use a resource.
2. **Hold and wait condition** – a process holding at least one resource is waiting to acquire additional resources held by other processes.
3. **No preemption condition** – a resource can be released only voluntarily by the process holding it, after that process has completed its task.
4. **Circular wait condition** – there exists a set of waiting processes.

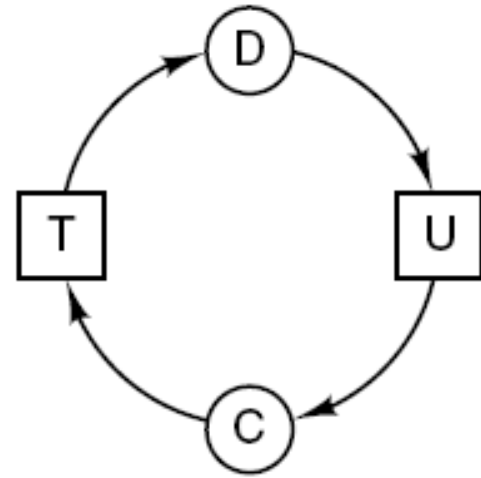
# Deadlock Modeling (1)



(a)



(b)



(c)

Resource allocation graphs.

(a) Holding a resource. (b) Requesting a resource. (c) Deadlock.

# Deadlock Modeling (2)

A  
Request R  
Request S  
Release R  
Release S

(a)

B  
Request S  
Request T  
Release S  
Release T

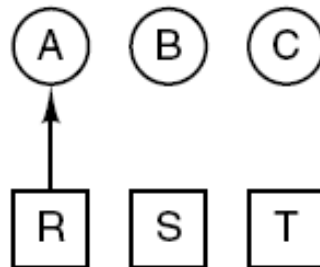
(b)

C  
Request T  
Request R  
Release T  
Release R

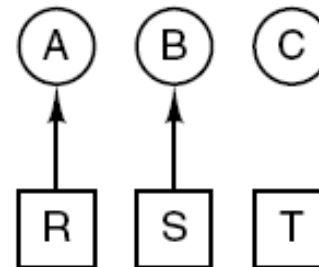
(c)

1. A requests R
2. B requests S
3. C requests T
4. A requests S
5. B requests T
6. C requests R  
deadlock

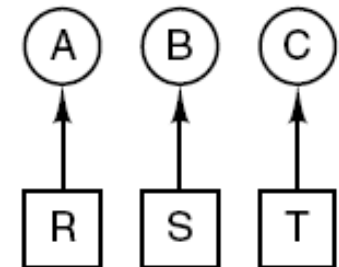
(d)



(e)



(f)



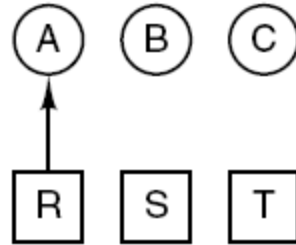
(g)

An example of how deadlock occurs  
and how it can be avoided.

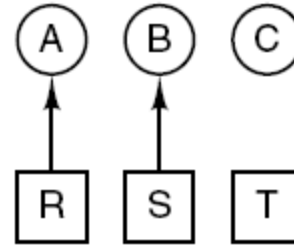
# Deadlock Modeling (3)

1. A requests R
2. B requests S
3. C requests T
4. A requests S
5. B requests T
6. C requests R  
deadlock

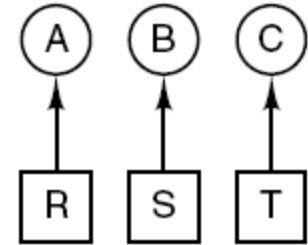
(d)



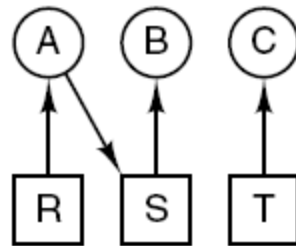
(e)



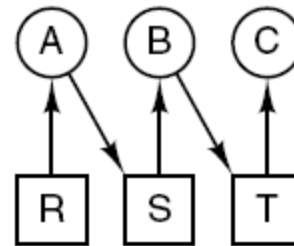
(f)



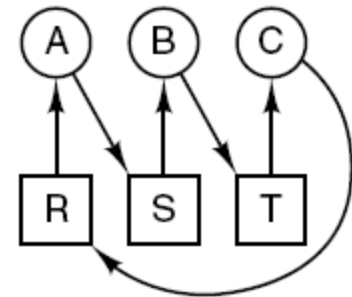
(g)



(h)



(i)



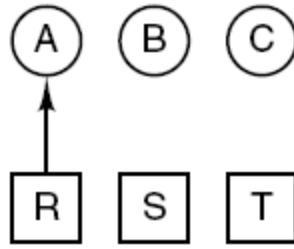
(j)

An example of how deadlock occurs  
and how it can be avoided.

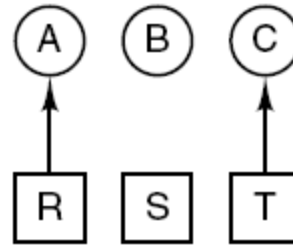
# Deadlock Modeling (4)

1. A requests R
2. C requests T
3. A requests S
4. C requests R
5. A releases R
6. A releases S  
no deadlock

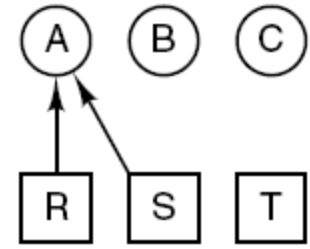
(k)



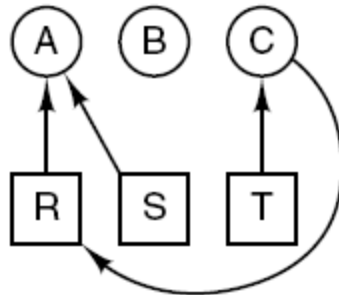
(l)



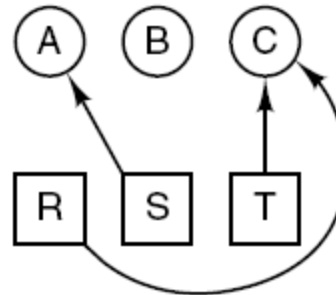
(m)



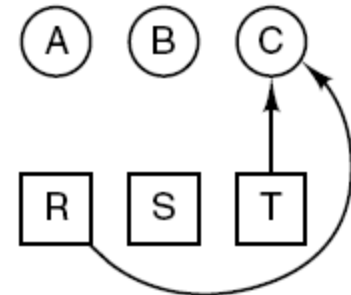
(n)



(o)



(p)



(q)

An example of how deadlock occurs  
and how it can be avoided.